

NURAi

Технический документ проекта

Интерактивный голосовой AI-помощник для O!Store

Мобильный оператор O! (Nur Telecom)

Бишкек, Кыргызстан

Автор: Виктор Ким, Android-разработчик

Целевое устройство: LG 65TR3DK-B (Android 13)

Документ предназначен для использования в качестве базы знаний RAG-системы AI Advent Challenge #8. Содержит полный обзор архитектурных решений, технологического стека, аппаратных требований, интеграций с внешними сервисами и открытых вопросов проекта.

Содержание

- Часть 1. Обзор проекта NURAI
- Часть 2. Эволюция требований проекта
- Часть 3. Аппаратная платформа LG 65TR3DK-B
- Часть 4. Микрофон и внешнее аудио-железо
- Часть 5. Kiosk mode и дизайн пространства
- Часть 6. Работа с 3D-аватаром (Avaturn)
- Часть 7. Рендер через SceneView и Filament
- Часть 8. Переход к видео-подходу
- Часть 9. State Machine и логика состояний
- Часть 10. Аудио пайплайн: VAD, AEC, потоки
- Часть 11. Сервисы STT и TTS
- Часть 12. Матрица регламентных ответов
- Часть 13. Контракты API с контакт-центром
- Часть 14. Безопасность и приватность
- Часть 15. Подходы к реализации Lip-Sync
- Часть 16. Стандарт виземы и маппинг фоном
- Часть 17. Защита от посторонних звуков
- Часть 18. Поиск ответов и HybridMatcher
- Часть 19. UI/UX дизайн интерфейса
- Часть 20. Многоязычность (6 языков)
- Часть 21. Технологический стек проекта
- Часть 22. Открытые вопросы для РМ
- Часть 23. Ключевые ошибки и уроки проекта

Часть 1. Обзор проекта NURAI

Проект NURAI — это интерактивный голосовой AI-помощник для торговых залов O!Store (мобильный оператор O! / Nur Telecom, Кыргызстан). Устройство представляет собой вертикальную интерактивную панель с цифровым аватаром, которая консультирует посетителей по тарифам, услугам eSIM, роумингу и другим вопросам, обычно решаемым сотрудниками магазина или контакт-центром.

Основная бизнес-цель проекта — снизить нагрузку на сотрудников магазина по типовым консультациям, сократить время ожидания клиента, повысить доступность информации об услугах O! и улучшить клиентский опыт в точке продаж.

Ожидаемый результат — в торговом зале O!Store размещается интерактивная панель с цифровым аватаром NURAI, которая принимает голосовые запросы клиента, автоматически определяет язык обращения из шести поддерживаемых (русский, кыргызский, английский, китайский, турецкий, немецкий) и выдаёт голосовой и текстовый ответ на основе актуальной базы знаний Контакт-центра.

В рамках проекта должны быть реализованы: голосовое взаимодействие клиента с AI-помощником, визуальный интерфейс панели с аватаром, интеграция с API Контакт-центра и базой знаний, поддержка регламентированных ответов на нестандартные вопросы, отображение ссылок и открытие WebView на панели, отчётность по обращениям клиентов.

Часть 2. Эволюция требований проекта

Изначально в проекте был реализован говорящий аватар на основе Lottie-анимации: подготовлен пайплайн TTS и STT, добавлен псевдо-репозиторий и API для коммуникации. После этого требования изменились — заказчик потребовал 3D-девушку по пояс, которая будет разговаривать с синхронной анимацией губ и двигаться как ассистент.

Первая попытка реализации была направлена на использование Ready Player Me (RPM) как источника 3D-модели вместе с движком SceneView (обёртка над Filament) на Android. Однако выяснилось, что RPM был приобретён Netflix в декабре 2025 и объявил о закрытии сервисов 31 января 2026 — что делает его непригодным для production. Альтернативным решением стал Avaturn — сервис с аналогичной технологией, поддерживающий ARKit blendshapes и viseme-морфтаргеты для lip-sync.

При работе с 3D-моделью также обнаружилась проблема лимита костей в Filament: скелет модели с более чем 256 костями (особенно с bone-based face rig) не рендерится корректно. Решение — использовать модели с blend shape (morph target) анимацией лица вместо костной системы, что и предлагает Avaturn при

выборе типа тела T2.

После оценки производительности целевого устройства и запроса заказчика на реалистичный образ ассистента было принято решение отказаться от 3D в пользу видео. Требования снова изменились: было принято решение показывать заранее подготовленное видео вместо 3D-аватара, что кардинально упростило клиентскую часть.

Финальным поворотом стало обсуждение AI-генерации видео через современные модели (Kling 3.0, Veo 3.1, Runway Gen-4.5), которые позволяют получить talking head с lip-sync на нескольких языках без физической съёмки актрисы.

Часть 3. Аппаратная платформа LG 65TR3DK-B

Целевое устройство проекта — LG 65TR3DK-B, интерактивная доска серии CreateBoard от LG. Изначально предполагалось, что это устройство подходит для рендера 3D-графики в реальном времени, однако анализ спецификаций показал существенные ограничения.

Реальные характеристики устройства (уточнённые по официальной странице LG):

- Процессор: Quad-core ARM Cortex-A55
- GPU: Mali G52 MP2
- ОЗУ: 8 ГБ
- Хранилище: 64 ГБ
- Операционная система: Android 13 (поверх webOS)
- Экран: 65 дюймов, разрешение 4K (3840×2160)
- Ориентация в проекте: вертикальная (портретная)
- Touch: мульти-тач 40 точек
- Сеть: Wi-Fi 6, Bluetooth 5.0, Gigabit LAN
- Аудио: динамики 15W × 2 (всего 30W)

Коммуникационные порты:

- HDMI ×3
- RGB/VGA
- Audio In (3.5мм миниджек)
- RS232C
- LAN (Gigabit)
- USB 3.0 Type A ×4
- USB 2.0 Type A ×1
- USB Type-C ×1
- Output: DP Out, Audio Out (Optical SPDIF), Touch USB ×2
- OPS slot для мини-ПК

Критические ограничения:

1. У устройства HET встроенного микрофона. Такая функция есть только у старшей модели 65TR3PN-B (CreateBoard Pro). Это blocker для голосового взаимодействия.
2. Процессор Cortex-A55 — четыре энергоэффективных ядра, по производительности сопоставимо с бюджетными смартфонами 2019 года. Не рассчитан на реалтайм 3D-рендер сложных персонажей.
3. GPU Mali G52 MP2 — мобильный класс, подходит для UI и видео, но не для реалистичного 3D персонажа с lip-sync.
4. Разрешение 4K требует в 4 раза больше вычислений чем FHD, что дополнительно усугубляет проблему рендера.

Для сравнения: средний игровой смартфон 2025 года имеет процессор в 5–8 раз мощнее по графической части. Реалистичный 3D-персонаж с лицевой анимацией — это нагрузка, сопоставимая с современной мобильной игрой.

Способы обойти отсутствие микрофона:

1. Внешний USB-микрофон (рекомендуется).
2. Аналоговый микрофон через Audio In (менее удобно из-за роутинга на Android).
3. OPS-модуль с собственным микрофонным входом (меняет всю архитектуру).

Часть 4. Микрофон и внешнее аудио-железо

Для работы голосового ассистента в шумном торговом центре критически важен качественный микрофон с направленным захватом. Требования ТЗ включают: конус захвата 90 градусов, дальность 0.5–1.5 метра, подавление боковых зон на 18–24 dB, отсеечение звуков за пределами 2 метров.

Варианты USB-микрофонов:

Бюджетный вариант (~\$30–50):

- Обычные USB-конференц-микрофоны от Logitech, Anker, Jabra Speak.
- Подходят для относительно тихой зоны.
- Без beamforming, ограниченное шумоподавление.

Оптимальный вариант (~\$70–150):

- ReSpeaker XVF3800 (Seeed Studio) — 4-микрофонная решётка с аппаратным beamforming, AEC, шумоподавлением.
- Audfly AI Directional Microphone Array — направленный захват в зоне 30 градусов, специально для AI-киосков.

Премиум вариант (~\$200–400):

- Shure MV5C, Yamaha YVC-200 — профессиональные конференц-микрофоны.

Рекомендация — ReSpeaker XVF3800 или аналогичный beamforming-микрофон. Аппаратный AEC на самом микрофоне даёт очищенный сигнал без нагрузки на CPU

устройства. Это особенно ценно на слабом железе Cortex-A55.

Программная работа с USB-микрофоном на Android 13:

```
val audioManager = getSystemService(Context.AUDIO_SERVICE) as AudioManager
val inputDevices = audioManager.getDevices(AudioManager.GET_DEVICES_INPUTS)

val usbMic = inputDevices.firstOrNull {
    it.type == AudioDeviceInfo.TYPE_USB_DEVICE ||
    it.type == AudioDeviceInfo.TYPE_USB_HEADSET
}

if (usbMic != null) {
    audioRecord.setPreferredDevice(usbMic)
}
```

Без явного указания `setPreferredDevice` Android может выбрать неверный источник звука или встроенный микрофон системы для служебных функций.

Проверка доступных аудио-устройств на устройстве:

```
val devices = audioManager.getDevices(AudioManager.GET_DEVICES_INPUTS)
devices.forEach { device ->
    Log.d("AudioCheck", "Product: ${device.productName}")
    Log.d("AudioCheck", "Type: ${describeType(device.type)}")
    Log.d("AudioCheck", "Sample rates: ${device.sampleRates.joinToString()}")
}

private fun describeType(type: Int): String = when (type) {
    AudioDeviceInfo.TYPE_BUILTIN_MIC -> "BUILTIN_MIC"
    AudioDeviceInfo.TYPE_USB_DEVICE -> "USB_DEVICE"
    AudioDeviceInfo.TYPE_USB_HEADSET -> "USB_HEADSET"
    AudioDeviceInfo.TYPE_WIRED_HEADSET -> "WIRED_HEADSET (3.5mm)"
    AudioDeviceInfo.TYPE_LINE_ANALOG -> "LINE_ANALOG (Audio In)"
    else -> "OTHER ($type)"
}
```

Часть 5. Kiosk mode и дизайн пространства

Для работы устройства в публичном месте необходим полноценный kiosk mode — приложение должно занимать весь экран без системных кнопок навигации, а пользователи не должны иметь возможность выйти из приложения или запустить другие приложения.

Три уровня скрытия системных элементов:

Уровень 1: Скрыть контролы плеера

```
PlayerView(ctx).apply {
    useController = false
    setShutterBackgroundColor(android.graphics.Color.TRANSPARENT)
    setKeepContentOnPlayerReset(true)
```

```
        resizeMode = AspectRatioFrameLayout.RESIZE_MODE_ZOOM
    }
```

Уровень 2: Полноэкранный режим Android

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        WindowCompat.setDecorFitsSystemWindows(window, false)

        WindowInsetsControllerCompat(window, window.decorView).apply {
            hide(WindowInsetsCompat.Type.systemBars())
            systemBarsBehavior =
                WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE
        }

        setContent { NurAiApp() }
    }
}
```

Уровень 3: Lock Task Mode (kiosk)

Вариант А — Lock Task без device owner (проще):

```
<activity
    android:name=".MainActivity"
    android:lockTaskMode="if_whitelisted">
</activity>
```

Вариант Б — Полноценный kiosk через Device Policy Manager:

Требуется чтобы приложение было Device Owner, что означает factory reset устройства и установку через ADB:

```
adb shell dpm set-device-owner com.yourcompany.nurai/.MyDeviceAdminReceiver
```

После этого можно заблокировать выход из приложения, системные жесты, установку других приложений, настроить автозапуск при включении устройства.

Для production на LG-стенде device owner режим обязателен.

Дизайн физического пространства также критичен:

- Расположение стенда в углу или нише (не посреди прохода) — стены отсекают шум
- Высота микрофона на уровне рта среднего взрослого (~1.5–1.6 м от пола)
- Маркировка пола — нарисованный круг "встаньте сюда" в 50–80 см от экрана
- Козырёк сверху над зоной микрофона отсекает звуки сверху
- Расположение подальше от рабочих мест продавцов (минимум 4–5 метров)

Часть 6. Работа с 3D-аватаром через Avaturn

Ранее проект пытался использовать 3D-аватара через связку Ready Player Me + SceneView. Однако в декабре 2025 года RPM был приобретён Netflix, и сервис объявил о полном закрытии 31 января 2026 года. Строить production-систему на сервисе, который прекратит работу через несколько месяцев, недопустимо: если CDN с аватарами будет отключён, приложение сломается.

После анализа альтернатив был выбран Avaturn — сервис с аналогичной архитектурой и API. Ключевые преимущества:

- Реалистичнее чем RPM (заметно ближе к фото)
- Активно развивается
- Поддерживает те же стандарты ARKit blendshapes и Oculus visemes
- Совместим с Mixamo анимациями
- Бесплатный тариф позволяет создать аватара и скачать GLB-файл

Критичный момент при работе с Avaturn — выбор типа тела:

- T1 body — самый реалистичный, но со статичным лицом (нельзя анимировать)
- T2 body — с отдельными глазами и ртом, поддерживает ARKit blendshapes и виземы, чуть менее похож на исходное селфи

Для NURAI необходим T2 body — без него lip-sync невозможен. При первой попытке скачать модель был выбран T1, и файл получился 4 МБ без морф-таргетов. После пересоздания с T2 файл стал 8–15 МБ и содержал полный набор блендшейпов.

Полный список морф-таргетов у типичной Avaturn T2 модели:

Виземы (15 штук по Oculus стандарту):

- viseme_sil, viseme_PP, viseme_FF, viseme_TH, viseme_DD
- viseme_kk, viseme_CH, viseme_SS, viseme_nn, viseme_RR
- viseme_aa, viseme_E, viseme_I, viseme_O, viseme_U

Отличие от стандартного Oculus: Avaturn использует I/O/U вместо ih/oh/ou для последних трёх виземы. Фонемы те же, только другая нотация. Маппинг 1:1.

ARKit blendshapes (52 штуки):

- Eyes: eyeBlinkLeft/Right, eyesClosed, eyeWideLeft/Right, eyeSquintLeft/Right
- Gaze: eyeLookUpLeft/Right, eyeLookDownLeft/Right, eyeLookInLeft/Right, eyeLookOutLeft/Right
- Brows: browInnerUp, browDownLeft/Right, browOuterUpLeft/Right
- Mouth: mouthSmile, mouthSmileLeft/Right, mouthFrownLeft/Right, mouthOpen, mouthClose
- Jaw: jawOpen, jawForward, jawLeft/Right
- Cheeks: cheekPuff, cheekSquintLeft/Right
- Nose: noseSneerLeft/Right
- Дополнительные: mouthFunnel, mouthPucker, mouthRollUpper/Lower, mouthShrugUpper/Lower, tongueOut

Модель Avaturn разбита на несколько мешей (голова, тело, зубы, глаза, ресницы). При применении весов морф-таргетов необходимо применять их ко всем мешам, где этот таргет существует. Иначе получится ситуация когда губы двигаются, а зубы остаются на месте.

Kotlin enum для маппинга Oculus виземы на Avaturn имена:

```
enum class Viseme(val morphTargetName: String, val oculusIndex: Int) {
    SIL("viseme_sil", 0),
    PP("viseme_PP", 1),
    FF("viseme_FF", 2),
    TH("viseme_TH", 3),
    DD("viseme_DD", 4),
    KK("viseme_kk", 5),
    CH("viseme_CH", 6),
    SS("viseme_SS", 7),
    NN("viseme_nn", 8),
    RR("viseme_RR", 9),
    AA("viseme_aa", 10),
    E("viseme_E", 11),
    IH("viseme_I", 12),    // Avaturn использует I вместо ih
    OH("viseme_O", 13),
    OU("viseme_U", 14);

    companion object {
        private val byIndex = entries.associateBy { it.oculusIndex }
        fun fromOculusIndex(index: Int): Viseme? = byIndex[index]
    }
}
```

Часть 7. Рендер через SceneView и Filament

Для рендера 3D-аватара на Android был выбран SceneView — обёртка над Filament engine от Google. Актуальная версия на момент разработки — 2.3.3. Поддержка Jetpack Compose из коробки.

Зависимости в build.gradle.kts:

```
dependencies {
    implementation("io.github.sceneview:sceneview:2.3.3")
}

android {
    defaultConfig {
        minSdk = 26
        ndk {
            abiFilters += listOf("arm64-v8a", "armeabi-v7a")
        }
    }

    androidResources {
        noCompress += listOf("filamat", "ktx", "glb")
    }
}
```

```
}
```

Основной компонент рендера аватара — AvatarRendererImpl. Использует Filament через SceneView, кэширует индексы морф-таргетов для быстрого обновления, применяет веса ко всем мешам одновременно.

```
class AvatarRendererImpl(
    private val context: Context,
    private val scope: CoroutineScope
) : AvatarController {

    private var sceneView: SceneView? = null
    private var modelInstance: ModelInstance? = null
    private val morphTargetMap = mutableMapOf<String, MutableList<Pair<Int, Int>>>()

    private val _isReady = MutableStateFlow(false)
    override val isReady = _isReady.asStateFlow()

    fun attachSceneView(view: SceneView) {
        sceneView = view
        configureSceneForLowEndDevice(view)
    }

    // Оптимизация под слабое железо
    private fun configureSceneForLowEndDevice(view: SceneView) {
        view.apply {
            setBackgroundColor(android.graphics.Color.parseColor("#1a1a2e"))

            view.view.apply {
                antiAliasing = View.AntiAliasing.NONE
                sampleCount = 1

                ambientOcclusionOptions = ambientOcclusionOptions.apply {
                    enabled = false
                }

                bloomOptions = bloomOptions.apply {
                    enabled = false
                }

                dynamicResolutionOptions = dynamicResolutionOptions.apply {
                    enabled = true
                    quality = View.QualityLevel.LOW
                }

                renderQuality = renderQuality.apply {
                    hdrColorBuffer = View.QualityLevel.LOW
                }
            }
        }
    }
}
```

Проблема лимита костей в Filament: SceneView через Filament ограничивает количество костей в скелете skinned mesh значением 256. Модели с bone-based face rig (типа MetaHuman, некоторые из Character Creator) легко превышают этот лимит только на голове, и при попытке уменьшить количество костей форма лица теряется.

Решения проблемы 256 костей:

1. Разделить меш на части (одна модель = один лимит 256 костей на каждый skinned mesh)
2. Bone reduction в Blender: удалить кости ног, twist bones для плеч, часть пальцев, недеформирующие кости
3. Использовать blend shapes вместо костей для лица (ARKit-52 стандарт)

Для NURAI выбран путь #3 — модель Avaturn T2 использует blend shapes для лица и имеет всего ~65 костей в скелете, что укладывается в лимит с большим запасом.

Оптимизации рендера под Cortex-A55 + Mali-G52:

- Рендер в 1080p с апскейлом до 4K через SurfaceView (в 4 раза меньше нагрузка чем нативный 4K)
- Отключены anti-aliasing, MSAA, SSAO, bloom
- HDR color buffer отключён
- Включено динамическое снижение разрешения при просадках FPS
- Target 30 FPS вместо 60
- Один directional light + IBL, никаких shadows реального времени
- Текстуры уменьшены до 1024×1024 для лица, 512 для остального
- Draco compression для GLB-файлов

При работе с телефоном на этапе разработки была замечена сильная перегрев устройства. На стационарной LG-доске с активным охлаждением и питанием от сети эта проблема не должна проявляться так остро.

Часть 8. Переход к видео-подходу

После оценки сложности реализации 3D-аватара с lip-sync на Cortex-A55 было принято решение перейти к видео-подходу. Финальные требования: аватар отображается через заранее подготовленные видеоролики, что кардинально упрощает клиентскую часть.

Преимущества видео-подхода:

1. Всё тяжёлое (рендер лица, lip-sync, синтез голоса) — на стороне сервера или при подготовке контента
2. Устройство только запускает запрос, принимает MP4 файл или стрим, воспроизводит видео
3. Нагрузка на устройство в 50 раз меньше чем 3D-аватар
4. H.264 1080p декодируется хардварно на Cortex-A55 без проблем
5. Качество живой актрисы > стилизованного 3D
6. Устройство не греется

Требования к видео файлам для качественного результата:

- Разрешение: 1920×1080 (не 4K — на Cortex-A55 4K рискован, а на 65-дюймовом экране визуальной разницы почти нет)
- Кодек: H.264 baseline или main profile для универсальности

- Контейнер: MP4 с moov atom в начале (флаг -movflags faststart в ffmpeg) — критично для прогрессивной загрузки
- Битрейт: 4-6 Mbps достаточно для talking head
- FPS: 25 или 30

Стратегия хранения видео:

На устройстве (обязательно):

- Все system_videos: idle, listening, thinking — должны запускаться мгновенно
- Топ-20 самых частых ответов (FAQ по тарифам, eSIM, роумингу)
- Все fallback-видео
- Все регламентные ответы из Приложения 1

На сервере/CDN (опционально):

- Редкие или длинные ответы
- Видео которые часто обновляются (промо-акции)
- Emergency mode промо-ролики

Стратегия обновления через WorkManager раз в час:

1. Клиент запрашивает manifest у сервера
2. Сервер возвращает список актуальных версий каждого видео
3. Клиент скачивает изменившиеся видео в фоне
4. Старые версии удаляются после успешной загрузки новых
5. При запуске приложения никогда не ждём сеть — играем то что есть локально

Расчёты по размеру и загрузке для 30 видео на 6 языках:

Разрешение	Битрейт	30 сек	30 видео × 6 языков
----- ----- ----- -----			
1080p	3 Mbps	11 МБ	1.98 ГБ
1080p	5 Mbps	19 МБ	3.42 ГБ
4K H.264	12 Mbps	45 МБ	8.1 ГБ
4K H.265	6 Mbps	22 МБ	3.96 ГБ

При 8 ГБ RAM и 64 ГБ хранилища на LG-доске места достаточно даже на 4K H.265 с 6 языками. Но 1080p H.264 остаётся оптимальным по производительности.

Часть 9. State Machine и логика состояний

Финальный дизайн state machine для NURAI включает 8 основных состояний, покрывающих весь пользовательский путь от ожидания до сложных случаев с webview и аварийным режимом. Реализация через sealed interface на Kotlin даёт компилятору полный контроль над обработкой всех состояний.

```
sealed interface NuraiState {
```

```

// === Стартовое состояние ===
data class Idle(
    val currentGreetingIndex: Int,    // 0..5 – язык приветствия
    val idleVideoPlaying: Boolean,
    val inviteCountdownMs: Long       // до приглашающего жеста
) : NuraiState

// === Активный диалог ===
data class Listening(
    val isHotMicMode: Boolean,         // постоянное слушание vs кнопка
    val partialTranscript: String,     // live превью
    val detectedLanguage: Language?,
    val silenceTimeoutMs: Long,        // до auto-stop (3-5 сек)
    val waveformAmplitude: Float
) : NuraiState

data class Processing(
    val query: String,
    val language: Language,
    val sessionId: String,
    val startedAt: Long,
    val timeoutMs: Long = 5000         // 5 сек до отказа по T3
) : NuraiState

data class Speaking(
    val answer: AnswerPayload,
    val isInterruptible: Boolean = true,
    val ttsPlaybackPositionMs: Long
) : NuraiState

// === Открыт WebView с сайтом компании ===
data class WebViewMode(
    val url: String,
    val inactivityTimeoutMs: Long = 30000,
    val lastInteractionAt: Long
) : NuraiState

// === Ошибки ===
data class Error(
    val errorCode: ErrorCode,
    val userMessage: String,
    val autoReturnAfterMs: Long = 5000
) : NuraiState

// === Аварийный режим ===
data object EmergencyMode : NuraiState

// === Языковой выбор ===
data class ManualLanguageSelection(
    val reason: LanguageSelectionReason,
    val previousState: NuraiState
) : NuraiState

// === Бездействие после ответа ===
data class PostAnswerWaiting(
    val previousAnswer: AnswerPayload,
    val timeoutToPromptMs: Long = 60000,
    val timeoutToIdleMs: Long = 120000
) : NuraiState
}

```

Дополнительные типы:

```

enum class Language(val code: String, val nativeName: String) {
    RU("ru-RU", "Русский"),
    KY("ky-KG", "Кыргызча"),
    EN("en-US", "English"),
    ZH("zh-CN", "中文"),
    TR("tr-TR", "Türkçe"),
    DE("de-DE", "Deutsch")
}

enum class ErrorCode {
    KC_TIMEOUT,           // контакт-центр не ответил за 5 сек
    KC_UNAVAILABLE,       // нет ответа от API
    STT_ERROR,            // не удалось распознать речь
    STT_LOW_CONFIDENCE,   // распознано с низким качеством
    TTS_ERROR,            // не удалось озвучить
    NETWORK_ERROR,        // нет интернета
    FALLBACK_NO_ANSWER,   // ответ не найден в БЗ
    PROFANITY_FILTER,     // нецензурная лексика
    PII_DETECTED          // персональные данные
}

data class AnswerPayload(
    val answerId: String,
    val type: AnswerType,
    val displayText: String,
    val speakingText: String,
    val language: Language,
    val ttsAudioUrl: String?,
    val attachments: List<Attachment>,
    val isLongAnswer: Boolean
)

sealed interface Attachment {
    data class InstructionList(
        val steps: List<String>,
        val footer: String?
    ) : Attachment

    data class Chip(val id: String, val label: String, val targetVideoId: String) : Attachment

    data class PromoBanner(val imageUrl: String, val title: String, val subtitle: String?) : Attachment

    data class IconCardGroup(val cards: List<IconCard>) : Attachment

    data class ImageGallery(val images: List<String>) : Attachment

    data class TariffCardGroup(val cards: List<TariffCard>) : Attachment

    data class LinkButton(val label: String, val url: String, val openInWebView: Boolean) : Attachment
}

```

Все таймауты собраны в один объект для удобства настройки:

```

object Timeouts {
    const val KC_API_TIMEOUT_MS = 5_000L           // ответ КЦ
    const val USER_RESPONSE_KPI_MS = 20_000L       // общий KPI (для алертинга)
    const val POST_ANSWER_WAIT_MS = 60_000L        // ждём следующий вопрос
    const val SESSION_FAREWELL_MS = 120_000L       // прощаемся и сбрасываем
    const val LISTENING_SILENCE_MS = 5_000L        // в режиме слушания
    const val WEBVIEW_INACTIVITY_MS = 30_000L      // в webview
    const val GREETING_ROTATION_MS = 3_000L       // смена приветствия в idle
    const val EMERGENCY_MODE_CHECK_MS = 30_000L    // проверка сети в emergency
}

```

Часть 10. Аудио пайплайн: VAD, AEC, потоки данных

Аудио пайплайн NURAI включает несколько последовательных этапов обработки: захват с микрофона, шумоподавление, VAD (детекция голосовой активности), эхоподавление, streaming STT, определение языка. Каждый компонент можно реализовать как отдельным алгоритмом, так и через готовые библиотеки.

Общая архитектура пайплайна:

```
[USB Beamforming Mic]
  ↓ аппаратная фильтрация (beamforming, AEC, noise suppression)
[AudioRecord (16kHz mono PCM)]
  ↓
[Программный AEC (WebRTC)] — если нет аппаратного
  ↓
[Adaptive Noise Gate] — динамический порог по фону
  ↓
[VAD Silero] — определяет речь vs шум
  ↓
[Streaming STT (Google Cloud / Yandex / Whisper)]
  ↓ partial results каждые 100ms
[Language Detector] — определяет язык по первым 2-3 сек
  ↓
[STT Final Result] → в state machine
```

Ключевой трюк для получения качественного звука на Android — использовать `AudioSource.VOICE_COMMUNICATION` вместо `MIC`. Этот audio source автоматически включает встроенные DSP-эффекты устройства: AEC, NS (Noise Suppression), AGC (Automatic Gain Control).

```
val audioRecord = AudioRecord.Builder()
    .setAudioSource(MediaRecorder.AudioSource.VOICE_COMMUNICATION)
    .setAudioFormat(AudioFormat.Builder()
        .setSampleRate(16000)
        .setEncoding(AudioFormat.ENCODING_PCM_16BIT)
        .setChannelMask(AudioFormat.CHANNEL_IN_MONO)
        .build())
    .setBufferSizeInBytes(bufferSize)
    .build()

// Дополнительно включаем эффекты явно
if (AcousticEchoCanceller.isAvailable()) {
    AcousticEchoCanceller.create(audioRecord.audioSessionId)?.enabled = true
}
if (NoiseSuppressor.isAvailable()) {
    NoiseSuppressor.create(audioRecord.audioSessionId)?.enabled = true
}
if (AutomaticGainControl.isAvailable()) {
    AutomaticGainControl.create(audioRecord.audioSessionId)?.enabled = true
}
```

Voice Activity Detection через Silero VAD:

```
implementation("io.github.gkononov.android-vad:silero:2.0.5")

class SmartVoiceActivityDetector {
    private val vad = SileroVad(
        sampleRate = 16000,
        frameSize = 512,
        mode = Mode.NORMAL
    )

    private var noiseFloor = 0.0
    private var voiceThreshold = 0.0

    fun calibrateNoise(audioBuffer: ShortArray) {
        val rms = calculateRms(audioBuffer)
        noiseFloor = rms
        voiceThreshold = noiseFloor * 3.0
    }

    fun isSpeech(audioBuffer: ShortArray): Boolean {
        val rms = calculateRms(audioBuffer)
        return vad.isSpeech(audioBuffer) && rms > voiceThreshold
    }

    private fun calculateRms(buffer: ShortArray): Double {
        var sum = 0.0
        for (sample in buffer) {
            sum += sample * sample.toDouble()
        }
        return sqrt(sum / buffer.size)
    }
}
```

Adaptive Noise Gate — постоянно замеряем фоновый уровень шума и подстраиваем порог:

```
class AdaptiveNoiseGate {
    private val noiseHistory = ArrayDeque<Double>(50)
    private var currentNoiseFloor = 0.0

    fun processFrame(amplitude: Double): Boolean {
        noiseHistory.addLast(amplitude)
        if (noiseHistory.size > 50) noiseHistory.removeFirst()
        currentNoiseFloor = noiseHistory.average()

        // Порог = фон + 12 dB
        val threshold = currentNoiseFloor * 4.0
        return amplitude > threshold
    }
}
```

В тихий час системы срабатывает на тихий голос, в час пик автоматически поднимает порог.

Detection перебивания во время говорения аватара:

```
class InterruptionDetector(
    private val vad: SileroVad,
    private val ttsPlayer: TtsPlayer
) {
    private var speechFramesCounter = 0
    private val INTERRUPTION_THRESHOLD_FRAMES = 5
}
```

```

suspend fun monitor(audioFlow: Flow<ShortArray>) {
    audioFlow.collect { frame ->
        val isSpeech = vad.isSpeech(frame)

        if (isSpeech) {
            speechFramesCounter++
            if (speechFramesCounter >= INTERRUPTION_THRESHOLD_FRAMES) {
                ttsPlayer.stop()
                speechFramesCounter = 0
                onInterruption()
            }
        } else {
            speechFramesCounter = 0
        }
    }
}

```

Важно: если у устройства плохой AEC и аватар громко говорит, TTS-выход может сам себя перебивать. Поэтому нужна минимальная задержка 1 сек после начала TTS до возможности перебивания.

Часть 11. Сервисы STT и TTS

Для проекта требуется streaming STT с поддержкой 6 языков (русский, кыргызский, английский, китайский, турецкий, немецкий) и качественный TTS. Ниже сравнение основных вариантов.

STT сервисы:

Google Cloud Speech-to-Text:

- Поддерживает все 6 нужных языков
- Auto language detection (можно указать список альтернативных)
- Streaming recognition с частичными результатами
- Стоимость: ~\$0.024/мин
- Качество кыргызского: среднее

Yandex SpeechKit:

- Отлично работает с русским и кыргызским
- Streaming
- ~1 рубль за минуту
- Китайский и турецкий — хуже

OpenAI Whisper API:

- Отлично знает все 6 языков (Whisper-large)
- ~\$0.006/мин (дешевле Google)
- Streaming через OpenAI realtime API
- Требуется стабильный интернет

Vosk (локально):

- Бесплатно, офлайн

- Качество ниже коммерческих
- Кыргызский слабо поддержан

Рекомендация: Google Cloud STT как основной + Vosk локально как fallback при потере интернета.

TTS сервисы:

Microsoft Azure TTS:

- Отдаёт SSML с visemeEvent событиями (15 виземы Oculus)
- Идеально для lip-sync
- ~\$16 за 1M символов
- Все 6 языков с нейросетевыми голосами

Amazon Polly:

- Отдаёт Speech Marks с фонемами и таймингами
- Тоже идеально для lip-sync
- ~\$4 за 1M символов

Google Cloud TTS:

- Нейросетевые голоса WaveNet/Neural2
- Все 6 языков
- Лучшее качество по мнению многих

Yandex SpeechKit TTS:

- Отличный русский и кыргызский
- Китайский слабее

ElevenLabs:

- Премиум качество
- Возможность кастомного клонирования голоса
- Дороже, ~\$5-99/мес

Использование Azure TTS для получения виземы:

```
suspend fun synthesizewithVisemes(text: String, language: String): SynthesisResult {
    val ssml = buildSsmlWithVisemes(text, language)
    val response = azureClient.speakSsml(ssml)

    return SynthesisResult(
        audioUrl = response.audioUrl,
        visemes = response.visemeEvents.map { event ->
            VisemeEvent(
                visemeId = event.visemeId,
                offsetMs = event.audioOffset / 10_000
            )
        }
    )
}

// SSML разметка отправляется как строка с тегами:
// <speak version=1.0>
//   <voice name=ru-RU-DariyaNeural>
//     <mstts:viseme type=redlips_front/>
//     [текст ответа]
```

```
// </voice>
// </speak>
```

Для NURAI предлагается такой стек:

- STT: Google Cloud (streaming + language auto-detect)
- TTS: Azure для регламентных ответов (бесплатный тип 500к символов/мес) с виземами
- Fallback: Vosk offline STT + Yandex TTS
- Для брендинга можно рассмотреть ElevenLabs с кастомным голосом NURAI

Часть 12. Матрица регламентных ответов из Приложения 1

Приложение 1 к ТЗ содержит матрицу ответов на нестандартные вопросы — 40 категорий регламентных ответов. Эти ответы должны срабатывать на клиенте до отправки запроса в контакт-центр — это экономит трафик, снимает нагрузку с КЦ, гарантирует единый стиль ответов и работает офлайн.

Полный список категорий из Приложения 1:

Блокирующие (высокий приоритет):

- Процедурный вопрос (нужно действие человека)
- Непонятный / Смешанный язык
- Оскорбления / Маты
- Про руководство и правление компании
- "О! — казахстанская компания?"
- Вопрос не про связь (рецепт борща и т.д.)

Информационные о тарифах:

- Какой тариф самый дешёвый
- Что лучше: тариф X или тариф Y
- Какой тариф самый лучший
- Услуги и тарифы других операторов
- Можно ли бесплатно пользоваться интернетом
- Почему интернет не работает, если не заплатил

Личные вопросы к AI:

- Вопросы личного характера
- "Ты человек?"
- "Ты меня любишь?"
- "Ты следишь за мной?"
- "Ты робот?"
- "Ты умеешь петь?"
- "Ты можешь стать моим другом?"
- "Ты умеешь хранить секреты?"

- "Ты умеешь шутить?"
- "Ты настоящая?"
- "Можно просто поговорить со мной?"
- "Кто тебя создал?"
- "Ты спишь?"

Физика связи (образовательные FAQ):

- Почему у границ ловит сеть соседей
- Почему интернет плохо ловит в лифте
- Почему TikTok не грузится на работе
- Интернет работает хуже, когда идёт дождь
- Почему интернет быстрее ночью
- Почему телефон не ловит в горах
- Почему интернет пропал после авиарежима
- Почему интернет не работает в самолёте
- Почему телефон горячий

Забавные и каверзные вопросы:

- Если съесть SIM-карту, номер сохранится?
- Можно ли заряжать телефон в микроволновке
- Можно скачать больше памяти
- Можно ли SIM-карту стирать в машинке
- Почему телефон не работает без зарядки
- Можешь позвонить кому-нибудь

Стилистика ответов — живая, человеческая, с эмодзи. Например, для "Если съесть SIM-карту, номер сохранится?" ответ: "В системе номер сохранится, и его можно восстановить в O!Store. Но пробовать SIM-карту на вкус мы точно не рекомендуем — это вредно для здоровья! 😊"

Классификация категорий по приоритетам:

```
enum class RegulatedAnswerCategory(val priority: Int) {
    // Высокий приоритет — блокирующие
    PROFANITY(10),           // Маты/оскорбления
    PERSONAL_DATA_REQUEST(10), // Просьбы выдать персональные данные

    // Средний приоритет — нестандартные
    COMPETITOR_QUESTIONS(5), // Про других операторов
    LEADERSHIP_QUESTIONS(5),  // Про руководство
    OFF_TOPIC(5),              // Не про связь

    // Низкий приоритет — забавные/каверзные
    PERSONAL_AI_QUESTIONS(2),  // "Ты человек?", "Ты меня любишь?"
    JOKES_AND_FUN(2),          // "Спой", "Расскажи шутку"
    LIFE_HACKS(2),             // SIM в стиральной машинке

    // Информационные FAQ
    NETWORK_PHYSICS(1),        // Почему в горах, в лифте

    UNKNOWN(0)
}
```

```

class RegulatedAnswerMatcher(
    private val knowledgeBase: List<RegulatedAnswer>
) {
    fun tryMatch(query: String, language: Language): RegulatedAnswer? {
        val normalized = normalize(query)

        // Сначала проверяем высокий приоритет
        knowledgeBase
            .sortedByDescending { it.category.priority }
            .forEach { answer ->
                if (answer.triggerPatterns[language]?.any { it.containsMatchIn(normalized) } == true) {
                    return answer
                }
            }
        return null
    }

    private fun normalize(text: String): String {
        return text.lowercase()
            .replace("ё", "е")
            .replace(Regex("[^а-яа-z0-9\\s]"), "")
            .replace(Regex("\\s+"), " ")
            .trim()
    }
}

```

Оптимизация озвучки: поскольку регламентные ответы фиксированные, их можно записать заранее (один раз через профессиональный TTS, сохранить mp3). Тогда получается лучшее качество голоса, нулевая latency, не платим за TTS API на каждый ответ.

40 ответов × 6 языков = 240 mp3 файлов общим объёмом ~50-100 МБ. Кладём в assets или скачиваем при первом запуске.

Часть 13. Контракты API с контакт-центром

Взаимодействие клиентского приложения с backend-инфраструктурой включает несколько API. Ниже приведены предполагаемые контракты (уточняются при согласовании с командой контакт-центра).

Основной endpoint: POST /api/v1/nurai/query

Request:

```

{
  "sessionId": "nurai-1735574400-abc123",
  "deviceId": "nurai-bishkek-store-01",
  "query": {
    "text": "сколько стоит безлимитный интернет",
    "language": "ru-RU",
    "sttConfidence": 0.87
  },
  "context": {

```

```

    "conversationHistory": [
      {"role": "user", "text": "..."},
      {"role": "assistant", "text": "..."}
    ],
    "userLanguage": "ru-RU",
    "deviceLocation": "Bishkek_0Store_1"
  },
  "clientMetadata": {
    "appVersion": "1.0.0",
    "timestamp": "2026-06-30T10:30:00Z"
  }
}

```

Response 200 (успешный ответ):

```

{
  "answerId": "ans-12345",
  "responseTimeMs": 1230,
  "answer": {
    "type": "TEXT_WITH_ATTACHMENTS",
    "displayText": "Безлимитный интернет 0! Прайм – 235 сомов на 4 недели",
    "speakingText": "У 0! есть несколько безлимитных тарифов. Самый популярный – 0! Прайм за 235 сомов",
    "language": "ru-RU",
    "ttsAudioUrl": "https://cdn.o.kg/tts/ans-12345-ru.mp3",
    "attachments": [
      {
        "type": "chip",
        "id": "tariff_prime",
        "label": "0! Прайм",
        "action": {"type": "FOLLOW_UP_QUERY", "value": "Расскажи про 0! Прайм"}
      },
      {
        "type": "link",
        "label": "Все тарифы на o.kg",
        "url": "https://o.kg/tariffs",
        "openMode": "WEBVIEW"
      }
    ],
    "isLongAnswer": false
  },
  "fallback": null
}

```

Response 200 (fallback):

```

{
  "answerId": "fb-001",
  "answer": null,
  "fallback": {
    "type": "NO_ANSWER_FOUND",
    "displayText": "К сожалению, я не нашла ответ. Обратитесь к сотруднику 0!Store",
    "speakingText": "Извините, я не могу ответить на этот вопрос",
    "ttsAudioUrl": "https://cdn.o.kg/tts/fallback-no-answer-ru.mp3"
  }
}

```

Response 503 (Timeout / Unavailable):

```

{
  "error": "SERVICE_UNAVAILABLE",
  "code": "KC_TIMEOUT",
}

```

```

    "fallback": {
      "displayText": "Извините, технические неполадки...",
      "ttsAudioUrl": "https://cdn.o.kg/tts/error-tech-ru.mp3"
    }
  }
}

```

Endpoint для отчётности: POST /api/v1/nurai/analytics/event

```

{
  "deviceId": "nurai-bishkek-store-01",
  "sessionId": "...",
  "eventType": "QUERY_SUCCESS | QUERY_FAIL | SESSION_START | SESSION_END | WEBVIEW_OPENED | ERROR",
  "timestamp": "...",
  "data": {
    "language": "ru-RU",
    "query": "...",
    "answerId": "...",
    "errorCode": "...",
    "durationMs": 1230,
    "userInterruptedAvatar": true,
    "selectedAttachmentType": "link"
  }
}

```

Endpoint синхронизации контента: GET /api/v1/nurai/content/manifest

```

{
  "version": "2026-06-30T08:00:00Z",
  "emergencyVideos": [
    {
      "id": "promo-summer-2026",
      "url": "https://cdn.o.kg/promo/summer-2026.mp4",
      "checksum": "sha256:...",
      "sizeBytes": 24500000,
      "expiresAt": "2026-07-31"
    }
  ],
  "languageAssets": {
    "ru-RU": {
      "greetingAudio": "https://cdn.o.kg/greetings/ru.mp3",
      "inviteAudio": "https://cdn.o.kg/invite/ru.mp3",
      "errorAudio": "https://cdn.o.kg/errors/tech-ru.mp3"
    }
  },
  "avatarVideos": {
    "idle": "https://cdn.o.kg/avatar/idle.mp4",
    "listening": "https://cdn.o.kg/avatar/listening.mp4",
    "processing": "https://cdn.o.kg/avatar/processing.mp4",
    "speaking_neutral": "https://cdn.o.kg/avatar/speaking.mp4"
  }
}

```

WorkManager раз в час дёргает manifest, сравнивает версии файлов, скачивает обновлённые в фоне.

Опциональный WebSocket для streaming взаимодействия:

```

wss://api.o.kg/nurai/stream
→ {"type": "audio_start", "sessionId": "...", "language": "ru-RU"}
→ {"type": "audio_chunk", "data": "base64...PCM"}
→ {"type": "audio_end"}

```

```

← {"type": "partial_transcript", "text": "сколько..."}
← {"type": "final_transcript", "text": "сколько стоит интернет"}
← {"type": "answer_chunk", "text": "...", "speakingText": "..."}
← {"type": "tts_audio_url", "url": "..."}

```

Это позволит резко снизить latency — начать обрабатывать ещё пока пользователь договаривает.

Часть 14. Безопасность, приватность, PII

Требования безопасности и приватности в ТЗ:

Запрет на сбор персональных данных:

- ИИ категорически запрещено записывать в виде чата, парсить и сохранять любые персональные данные
- Номера телефонов, ПИН-коды, если клиент продиктовал их голосом — блокируются на лету
- Требование Антифрода

Политика хранения аудио:

- Голосовые файлы (записи речи) клиентов НЕ сохраняются в системе
- Разрешение на хранение аудио у клиентов не запрашивается
- В CRM уходит исключительно текстовая транскрибация диалога

PII Detection на клиенте — мягкая UX-фильтрация:

```

class ClientPiiFilter {
    // Регулярка для телефонов: +?ddd[-\s]?ddd[-\s]?ddd,?
    private val phoneRegex = Regex(PHONE_PATTERN)
    private val pinHintRegex = Regex(PIN_PATTERN, RegexOptions.IGNORE_CASE)

    fun containsPii(text: String): Boolean {
        return phoneRegex.containsMatchIn(text)
            || pinHintRegex.containsMatchIn(text)
    }

    fun maskPii(text: String): String {
        return text
            .replace(phoneRegex, "[НОМЕР]")
            .replace(pinHintRegex, "[ПИН]")
    }

    companion object {
        // Матчит форматы: +996XXX XXX XXX, 8XXXXXXXXXX и т.д.
        const val PHONE_PATTERN = "\\+?\\d{3,}\\s?\\d{3}\\s?\\d{3,}"
        // Матчит: "мой пин 1234", "код 12345" и т.д.
        const val PIN_PATTERN = "(мой|пин|pin|код)\\s*[:\\-]?\\s*\\d{4,6}"
    }
}

```

Если в STT-результате обнаружен такой паттерн — не отправляем на сервер, сразу даём ответ из регламентных: "Я не могу обрабатывать персональные данные. Пожалуйста, обратитесь к сотруднику O!Store с этим вопросом."

В CRM отправляется только маскированный текст для аналитики.

Жёсткая фильтрация делается на бэкенде. Это правильное разделение ответственности.

Фильтрация речи и нестандартные сценарии:

- Жёсткие фильтры защиты от нецензурной лексики
- Провокационных запросов
- Попыток некорректного использования AI

В случае фиксации оскорблений, матов, каверзных вопросов про компанию или шуток, ИИ строго использует регламентированные ответы из Приложения 1.

WebView Whitelist для сайтов компании:

Основная защита — на бэке, он отдаёт только проверенные URL в поле url ответа. Клиент их открывает. Дополнительно, второй уровень защиты на клиенте:

```
val allowedDomains = listOf("o.kg", "mobile.o.kg", "pay.o.kg")

webView.webViewClient = object : WebViewClient() {
    override fun shouldOverrideUrlLoading(
        view: WebView?,
        request: WebResourceRequest?
    ): Boolean {
        val url = request?.url?.toString() ?: return true
        val host = Uri.parse(url).host ?: return true

        val isAllowed = allowedDomains.any { domain ->
            host == domain || host.endsWith(".$domain")
        }

        if (!isAllowed) {
            analyticsRepository.logBlockedRedirect(url)
            return true
        }
        return false
    }
}
```

Таймаут бездействия на сайте: если пользователь перешёл по ссылке на сайт и не проявляет активности, ровно через 30 секунд сессия принудительно сбрасывается, сайт закрывается, а система возвращается на главный экран ожидания для нового клиента.

Требования к направленности луча захвата звука (аппаратные):

- Подавление боковых зон на 18–24 dB
- Отсечение звуков вне 90-градусного конуса
- Эффективный захват на расстоянии 0.5–1.5 метров
- Голос дальше 2 метров воспринимается как фоновый шум

Это невозможно чисто программно — требует beamforming-микрофон.

Реализация акустического эхоподавления:

1. Интеграция опорного сигнала: звуковой поток от TTS дублируется в плагин эхоподавления как эталон
2. Адаптивная цифровая фильтрация: алгоритм в реальном времени сравнивает входящий сигнал с опорным
3. Всё что совпадает с эталоном (голос NURAI) вычитается из входящего аудиопотока
4. При перебивании алгоритм разделяет аудиопотоки

Часть 15. Подходы к реализации Lip-Sync

Реализация lip-sync для NURAI имеет несколько принципиально разных подходов, каждый со своими преимуществами и ограничениями.

Подход А: Pre-recorded видео полных ответов

Записываем актрису произносящей каждый ответ целиком. 40 регламентных ответов из Приложения 1 + топ-30 FAQ по тарифам = ~70 видео. Идеальное качество lip-sync (живой человек), но требует физической съёмки актрисы в студии, работу с многоязычной актрисой (либо разные актрисы на 6 языков), и не покрывает динамические ответы из БЗ.

Стоимость: \$3000-5000 на съёмку.

Подход В: Viseme-based склейка на сервере

Записываем актрису произносящей 15 базовых виземы (форм рта) + idle-выражения (5-7 вариантов). На сервере при подготовке ответа: TTS → JSON с виземами и таймингами → ffmpeg собирает видео из кусочков → готовый MP4 клиенту.

Что снимать у актрисы:

- 15 виземы (sil, PP, FF, TH, DD, kk, CH, SS, nn, RR, aa, E, I, O, U) — каждая 0.3-0.5 сек
- 4-5 idle-выражений — по 10-15 сек loop
- 4 "говорящих поз" тела (calm, engaged, explaining, friendly) — по 15 сек loop
- Приглашающий жест, прощание, thinking — короткие клипы
- Итого 1.5-2 часа съёмки в студии

Плюсы:

- Одна библиотека виземы работает на все 6 языков
- Гибкость: любой текст → видео
- Меньше съёмки чем при варианте А

Минусы:

- Нужен сервис TTS с виземами (Azure или Amazon Polly)
- Работа на сервере (ffmpeg + библиотека)

- Возможны артефакты на переходах между виземами

Стоимость: \$2000–3000 на съёмку + разработка серверной склейки.

Подход C: Real-time AI talking head (D-ID, HeyGen, Tavus)

Серверные сервисы принимают одно reference фото актрисы + текст → стримят готовое видео с lip-sync в реальном времени.

- D-ID Live Portrait: \$30–300/мес, latency 1–2 сек
- HeyGen Interactive Avatar: премиум качество
- Tavus Conversational Video: есть SDK

Плюсы:

- Любой динамический текст → видео с lip-sync
- Не нужна физическая съёмка
- Одно фото = бесконечно говорящих видео

Минусы:

- Требует интернет
- Ежемесячная плата
- Latency добавляется к общему ответу

Подход D: AI-генерация полных talking head видео (Kling 3.0, Veo 3.1)

По состоянию на 2026 год AI-модели генерации видео совершили качественный скачок. Топ-модели:

Kling 3.0 (февраль 2026):

- Native 4K, multilingual lip-sync
- Multi-shot storyboard с character consistency через всю сессию
- ~\$0.10/сек
- Free tier с ограничениями

Veo 3.1 (Google):

- Единственная модель с 48kHz синхронизированной речью
- Native audio
- \$0.15–0.50/сек
- Лучший all-rounder

Runway Gen-4.5:

- Reference images + motion brush для character consistency
- Отличный control surface
- \$12–95/мес

Seedance 2.0 (ByteDance):

- Motion-first generation
- Reference video для определения движений персонажа

HappyHorse-1.0 (Alibaba, апрель 2026):

- 7-language lip-sync (English, Mandarin, Cantonese, Japanese, Korean, German, French)

- Топ по качеству

Sora 2 (OpenAI) — устарел:

- Web/app отключены 26 апреля 2026
- API отключается 24 сентября 2026
- НЕ рекомендуется для новых проектов

Расчёт стоимости для NURAI через Kling 3.0:

- 40 регламентных ответов × 15 сек × \$0.10 = \$60
- 10 idle/listening/thinking клипов × 10 сек × \$0.10 = \$10
- 30 FAQ видео × 15 сек × \$0.10 = \$45
- Итого начальная библиотека: ~\$115

Это в 30–50 раз дешевле студийной съёмки.

Workflow AI-генерации:

1. Один раз: генерим reference image актрисы в Midjourney/Flux
2. Загружаем reference в Kling 3.0 или Runway Gen-4.5
3. Для каждого ответа: текст + reference + промпт → готовое MP4 с lip-sync
4. Складываем в assets или CDN

Важно: character consistency между генерациями — критическая проблема. Даже с reference image разные генерации дают микро-различия (цвет губ, форма носа, освещение). Для последовательного проигрывания видео это ок, но для viseme-склейки не подойдёт (стыки будут видны).

Проблема с кыргызским: большинство моделей обучены на английском, китайском, европейских языках. Кыргызский может работать плохо в lip-sync. Требуется тестирования.

Часть 16. Стандарт виземы и маппинг фонем

Понимание виземы важно для нескольких подходов. Визема — это визуальная форма рта, соответствующая произнесению определённого звука (фонемы). 15 стандартных виземы (Oculus Lipsync стандарт) покрывают все основные фонемы почти всех языков.

Стандартный набор Oculus Lipsync visemes:

Визема	Английские звуки	Русские аналоги	Что делает рот
-----	-----	-----	-----
sil	тишина	-	Закрытый рот, нейтрально
PP	p, b, m	П, Б, М	Плотно сжатые губы
FF	f, v	Ф, В	Нижняя губа к верхним зубам
TH	th	-	Язык между зубами

DD	t, d, n	Т, Д, Н	Кончик языка к верхним зубам
kk	k, g	К, Г, Х	Задняя часть языка поднята
CH	ch, sh, zh	Ч, Ш, Щ, Ж	Губы округлены, выдвинуты
SS	s, z	С, З, Ц	Губы растянуты, зубы сжаты
nn	n, l	Н, Л	Кончик языка у верхнего нёба
RR	r	Р	Язык вибрирует
aa	ah	А	Широко открытый рот
E	eh	Э, Е	Рот средне-открытый
ih/I	ih	И, Й	Губы растянуты в улыбку
oh/O	oh	О, Ё	Губы округлены в "О"
ou/U	oo	У, Ю	Губы вытянуты трубочкой

Как это работает при синтезе речи:

1. TTS озвучивает текст → получаем mp3 + список фонем с таймингами
2. Маппим фонемы на виземы (один-к-одному словарь)
3. Получаем список [(viseme, startMs, durationMs), ...]
4. На каждый момент видео = клип соответствующей виземы
5. Между кадрами — плавный переход

Пример для фразы "Привет":

- 00-80ms: viseme_PP (П)
- 80-180ms: viseme_RR (Р)
- 180-320ms: viseme_I (И)
- 320-450ms: viseme_FF (В)
- 450-600ms: viseme_E (Е)
- 600-750ms: viseme_DD (Т)

Где брать тайминги фонем:

Вариант А — TTS отдаёт виземы сразу (Azure, Amazon Polly):

```
// Формируем SSML разметку для Azure TTS
val ssml = buildAzureSsmlRequest(
    voiceName = "ru-RU-DariyaNeural",
    text = text,
    includeVisemes = true
)

// SSML содержит теги:
//   speak version=1.0 xmlns:mstts=https://www.w3.org/2001/mstts
//     voice name=ru-RU-DariyaNeural
//       mstts:viseme type=redlips_front
//       текст ответа
//     /voice
//   /speak

val response = azureClient.speakSsml(ssml)
// response.visemeEvents содержит список [(visemeId, offsetMs), ...]
```

Вариант В — анализ аудио постфактум:

- Montreal Forced Aligner
- Wav2Vec2 phoneme model

- Oculus Lipsync SDK

Вариант С — G2P преобразование до TTS:

- eSpeak NG (100+ языков включая кыргызский)
- Phonemizer

Рекомендация — Azure TTS с виземами (входит в бесплатный тир 500k символов/мес).

Мультиязычность — одна библиотека на все 6 языков:

- Русский/кыргызский — стандартные 15 виземы + RR хорошо работает на "P"
- Английский — стандартные 15
- Немецкий — может потребоваться umlaut для Ö/Ü, но O/U достаточно
- Турецкий — стандартные 15
- Китайский — тоновая система, но виземы те же (тон — это аудио)

Одна и та же библиотека из 15 видео работает на все 6 языков. Это огромный плюс viseme-подхода.

Часть 17. Защита от посторонних звуков

ТЗ требует чтобы NURAI реагировала только на людей перед панелью, а не на посторонние голоса из ТРЦ. Это одна из самых сложных инженерных задач проекта. В индустрии известно что conversational AI трудно работает в местах высокого трафика.

Комбинация методов для NURAI:

Метод 1: Beamforming микрофонная решётка (основной)

Направленный микрофон с DoA (Direction of Arrival) и beamforming фокусируется на звуке с определённого направления, подавляя шум с других. ReSpeaker XVF3800 или Audfly AI Directional Microphone Array.

Метод 2: Push-to-talk активация

Самый надёжный способ определить "кто говорит" — тот кто нажал кнопку. В ТЗ уже реализовано: пользователь нажимает кнопку микрофона для запуска сессии. В сессии идёт постоянное слушание до таймаута.

Метод 3: Camera-based face detection

USB-камера (\$20-30) + Mediapipe Face Detection: активируем listening только когда в кадре лицо близко к камере.

```
val faceDetector = FaceDetector.builder()
    .setModelAssetPath("face_detection_short_range.tflite")
```

```

        .build()

cameraFrameFlow.collect { frame ->
    val faces = faceDetector.detect(frame)
    val nearFace = faces.firstOrNull { it.boundingBox.width() > MIN_FACE_WIDTH }
    isUserPresent = nearFace != null
}

```

Дополнительный бонус: можно детектить взгляд в камеру. Активируем только когда пользователь смотрит на экран.

Метод 4: Speaker identification (фиксация голоса)

После первой фразы пользователя система запоминает voice fingerprint. Все остальные голоса в текущей сессии игнорируются до конца сессии.

Технологии:

- 3D-Speaker от Alibaba
- WeSpeaker
- Resemblyzer Lite на TFLite

Модели 5–10 МБ, работают локально на устройстве.

Метод 5: Adaptive SNR threshold

Постоянно замеряем фоновый уровень шума (RMS за последние 5 сек). Активируем STT только когда новая речь громче фона на 12–15 dB.

```

class AdaptiveNoiseGate {
    private val noiseHistory = ArrayDeque<Double>(50)
    private var currentNoiseFloor = 0.0

    fun processFrame(amplitude: Double): Boolean {
        noiseHistory.addLast(amplitude)
        if (noiseHistory.size > 50) noiseHistory.removeFirst()
        currentNoiseFloor = noiseHistory.average()

        val threshold = currentNoiseFloor * 4.0 // 12 dB = 4x amplitude
        return amplitude > threshold
    }
}

```

В тихий час срабатывает на тихий голос, в час пик автоматически поднимает порог.

Метод 6: Distance sensor / ToF proximity

Внешний ToF (Time of Flight) сенсор — дешёвый (\$5–15) USB-сенсор измеряет дистанцию до объекта перед панелью. Активация слушания только когда кто-то ближе 1.5 метра.

- VL53L1X — ToF до 4 метров
- Стандарт в банкоматах и автоматах продажи

Метод 7: Дизайн физического пространства

Не код, но критически важно:

- Расположение стенда в углу или нише — стены отсекают шум
- Звукопоглощающие материалы на ближайших стенах
- Высота микрофона на уровне рта среднего взрослого
- Маркировка пола "встаньте здесь"
- Козырёк сверху над зоной микрофона
- Расположение подальше от рабочих мест продавцов

Рекомендуемая комбинация для NURAI:

Оптимум:

- ReSpeaker XVF3800 или аналог (\$70–100)
- USB веб-камера для face detection (\$20–30)
- Speaker identification (бесплатно, локально)
- Push-to-talk активация (уже в UI)
- Дизайн пространства (угловое расположение)

Стоимость дополнительного железа: \$90–130 за стенд.

Разработка: 3–4 дня.

Без beamforming-микрофона ТЗ-требования физически невыполнимы. Это блокер для голосового флоу.

Часть 18. Поиск ответов и HybridMatcher

Поиск нужного ответа при получении текстового запроса от пользователя — критически важный компонент. Пример: пользователь говорит "Не могу войти в аккаунт", а нужное видео называется "Что делать, если забыли пароль". Простой keyword matching здесь не работает.

Три уровня решения:

Уровень 1: Расширенные ключевые слова (минимальный)

Под каждое видео/ответ пишутся все мыслимые формулировки:

```
{
  "id": "forgot_password",
  "title": "Что делать если забыли пароль",
  "keywords": [
    "забыл пароль", "забыла пароль", "не помню пароль",
    "восстановить пароль", "сбросить пароль"
  ],
  "trigger_phrases": [
    "не могу войти", "не могу зайти", "не пускает",
    "не могу авторизоваться", "не получается войти",
    "проблема со входом", "ошибка входа", "вход не работает",
    "почему не пускает", "не входит в аккаунт",
    "не открывается аккаунт", "не заходит в личный кабинет"
  ]
}
```

```
    }
}
```

Разделение на keywords (прямые упоминания) и trigger_phrases (косвенные — "проблема которую решает это видео") — правильно методологически. 15–30 вариантов покрывают 80% запросов.

Уровень 2: Fuzzy matching

Добавляем нормализацию + fuzzy matching через расстояние Левенштейна или Jaro-Winkler.

```
implementation("me.xdrop:fuzzywuzzy:1.4.0")

class FuzzyMatcher {
    fun findBest(query: String, catalog: List<VideoEntry>): VideoEntry? {
        val normalizedQuery = normalize(query)
        var bestEntry: VideoEntry? = null
        var bestScore = 0

        catalog.forEach { entry ->
            entry.keywords.forEach { keyword ->
                val score = FuzzySearch.tokenSetRatio(
                    normalizedQuery,
                    normalize(keyword)
                )
                if (score > bestScore) {
                    bestScore = score
                    bestEntry = entry
                }
            }
        }

        return if (bestScore >= 60) bestEntry else null
    }
}
```

Уровень 3: Semantic search через embeddings

Считаем векторные представления каждой trigger_phrase + вектор вопроса пользователя, ищем ближайшие по cosine similarity.

Локально через ONNX Runtime:

- Модель: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
- Размер модели: ~120 МБ
- Поддерживает русский
- Векторы фраз каталога посчитать заранее на сервере
- В рантайме считаем только вектор вопроса пользователя

Через серверный API:

- OpenAI embeddings
- Cohere
- Свой бэкенд с sentence-transformer

Для 10–30 видео на слабом железе — embeddings избыточно. Уровня 2 достаточно. Уровень 3 имеет смысл при 100+ записей.

Гибридный подход для NURAI (рекомендация):

```
class HybridMatcher {
    fun findBest(query: String, catalog: List<VideoEntry>): MatchResult {
        val normalized = normalize(query)

        // Уровень 1: точные фразы
        val exactMatch = findExact(normalized, catalog)
        if (exactMatch != null) {
            return MatchResult(exactMatch, confidence = 1.0f, method = "exact")
        }

        // Уровень 2: fuzzy matching
        val fuzzyMatch = findFuzzy(normalized, catalog)
        if (fuzzyMatch != null && fuzzyMatch.second >= FUZZY_THRESHOLD) {
            return MatchResult(
                fuzzyMatch.first,
                fuzzyMatch.second / 100f,
                "fuzzy"
            )
        }

        return MatchResult(null, 0f, "none")
    }

    private fun normalize(text: String): String {
        return text.lowercase()
            .replace("ё", "е")
            .replace(Regex("[^а-яа-z0-9\\s]"), "")
            .replace(Regex("\\s+"), " ")
            .trim()
    }

    companion object {
        const val FUZZY_THRESHOLD = 65
    }
}

data class MatchResult(
    val entry: VideoEntry?,
    val confidence: Float,
    val method: String
)
```

Логирование неуспешных запросов критично для итераций — какие вопросы задавали, на которые матчер не ответил. Это позволяет добавлять эти фразы в `trigger_phrases` и улучшать качество без переобучения моделей.

Часть 19. UI/UX дизайн интерфейса

Дизайн интерфейса NURAI реализуется как система состояний, где каждый экран соответствует определённому этапу диалога. Устройство используется вертикально.

Экраны главного флоу:

1. Главная (Idle):

- Аватар в полный рост, статичный
- Цикличная смена приветствий на 6 языках (шаг 2.5–3 сек)
- Плавный fade-out/fade-in 300–500 мс
- Полный цикл из 6 языков — 15–18 сек
- Кнопки внизу: Информация (?), Начать разговор (микрофон), Смена языка
- Сверху быстрые тематические чипсы: Подобрать тариф, Завести eSIM, Роуминг, Приложение

Тексты приветствий:

- Hello! I'm NURAI, your digital assistant at O!
- Салам! Мен — NURAI, O! санариптик жардамчыңызмын.
- Привет! Я — NURAI, ваш цифровой помощник O!
- 🇷🇺🇷🇺🇷🇺 NURAI🇷🇺🇷🇺 O! 🇷🇺🇷🇺🇷🇺
- Merhaba! Ben NURAI, O! dijital yardımcınızım.
- Hallo! Ich bin NURAI, Ihr digitaler Helfer von O!

2. Выбор языка (LanguageSelection):

- Модальная панель снизу с 5 языками
- Кнопка Продолжить
- Idle-видео продолжает играть на фоне

3. Инфо (Info):

- Описание NURAI
- Список действий: Выберите язык, Начните задавать вопрос, Завершите запись, История чата
- Кнопка Заккрыть внизу

4. Слушаю (Listening):

- Крупный план ассистента (голова/шея) + анимированное кольцо
- Внизу анимация звуковой волны (реактивная на амплитуду)
- Текст live-транскрипции ("Мне нужен пакет где есть безлимитный интернет...")
- Кнопки: Завершить, Слушаю (микрофон активен), История чата

5. Обработка (Processing):

- Крупный план, зацикленное дыхание
- Круговой лоадер
- Статус: "Ищу информацию..."
- SLA до 20 секунд, timeout 5 сек

6. Ответ (Speaking):

- Активная анимация (мимика, голос)
- Текст ответа мгновенно, единым полотном
- Attachments: чипсы, карточки, ссылки
- Кнопки: Завершить, NURAI говорит, История чата

7. История чата (ChatHistory):

- Список предыдущих сообщений сессии
- Карточки тарифов с подробностями
- Кнопки: Завершить разговор, Продолжить

Типы attachments (из анализа Figma):

Тип 1: InstructionList (нумерованный список шагов)

- Например: "Как подключить eSIM"
- 7 пронумерованных шагов
- Опционально футер "Стоимость: Бесплатно"

Тип 2: ChipGroup (выбор категории)

- 4-8 чипсов
- Например: Интернет, Звонки, Роуминг, O!TV и онлайн-кинотеатры

Тип 3: PromoBanner (изображение-баннер)

- Большая картинка с акцией
- Заголовок и подзаголовок

Тип 4: IconCardGroup (карточки с иконками)

- 2-4 карточки
- Например: Управлять тарифом, Контролирование номеров, Получать бонусы

Тип 5: ImageGallery (горизонтальная лента скриншотов)

- Например: скриншоты приложения "Мой O!"
- Скроллится вбок

Тип 6: TariffCardGroup (карточки тарифов)

- Название тарифа
- Цена
- Особенности (безлимит, минуты, каналы, соцсети)

Тип 7: LinkButton (переход на сайт)

- Открывает встроенный WebView с сайтом o.kg

Общие элементы всех экранов:

- Логотип NURAI + O! сверху
- Внизу: анимация звуковой волны + 3 кнопки

Особенности вёрстки:

- Видео занимает верхнюю часть экрана (~60%)
- Текст ответа и attachments — под видео
- Кнопки навигации — в самом низу
- Единый портретный layout под 65-дюймовый вертикальный экран

Тип отображения текста: текст ответа выводится мгновенно, единым полотном (без эффекта караоке или побуквенного телетайпа). Главный фокус — на аватаре и кликабельных плашках.

Часть 20. Многоязычность (6 языков)

Требование ТЗ — поддержка 6 языков: русский, кыргызский, английский, китайский, турецкий, немецкий. Это одно из самых нетривиальных требований к системе.

Автоматическое определение языка:

При возникновении сложностей с распознаванием речи (особенно кыргызского) система мгновенно выводит панель с ручным выбором языка.

Уровни определения:

Уровень 1: Language ID до отправки в STT

Определяем язык по первым 2–3 секундам аудио, чтобы выбрать правильную STT-модель.

Варианты:

- fastText langid — локально, ~100 МБ, поддерживает 176 языков
- Google Cloud Speech с alternativeLanguageCodes
- Whisper API с автоопределением

Уровень 2: STT с auto-detection

Google Cloud Speech-to-Text поддерживает alternativeLanguageCodes — указываем список альтернативных языков, модель сама выбирает наиболее вероятный.

```
val config = RecognitionConfig.newBuilder()
    .setLanguageCode("ru-RU")
    .addAllAlternativeLanguageCodes(listOf("ky-KG", "en-US"))
    .build()
```

Уровень 3: Ручной выбор языка

Bottom sheet с 5 языками + кнопка Продолжить. Триггеры:

- Пользователь тапнул кнопку "Смена языка"
- STT не смогло уверенно определить язык (confidence < 0.6)
- STT определило язык неправильно (пользователь сказал что не так)

```
enum class LanguageSelectionReason {
    USER_REQUESTED,        // пользователь тапнул кнопку
    FAILED_TO_DETECT,       // STT не смогло определить
    LOW_CONFIDENCE,         // низкая уверенность STT
    USER_CORRECTION        // пользователь поправил
}
```

Проблемы с кыргызским:

Кыргызский — один из самых сложных для голосовых технологий:

- Google Cloud STT поддерживает, но качество среднее
- Yandex — лучше, но всё равно не идеально
- Whisper — знает, но чувствителен к акценту
- Vosk — есть модель, но качество низкое

Рекомендации:

- Использовать Yandex SpeechKit для кыргызского
- Google Cloud для остальных языков
- Fallback на ручной выбор если confidence < 0.6

Мультиязычные TTS:

- Google Cloud TTS — все 6 языков, разное качество
- Azure TTS — есть кыргызский с 2024 года
- Yandex — топ для русского и кыргызского

Регламентные ответы на 6 языков:

40 ответов × 6 языков = 240 текстовых вариантов. Нужно решить:

- Автоперевод (быстро, но может потерять стилистику)
- Ручной перевод с адаптацией под культуру (правильнее, дороже)

Например, "О! — казахстанская компания?" — этот вопрос имеет смысл только на русском/кыргызском (национальный контекст). На немецком/китайском формулировка ответа должна быть другой.

Приоритеты по языкам:

1. Русский — основной
2. Кыргызский — обязательный (национальный)
3. Английский — для иностранцев
4. Турецкий — для турецкого бизнеса в Бишкеке
5. Немецкий — редко
6. Китайский — для китайских туристов и бизнеса

TTS-озвучка на 6 языков:

Регламентные ответы можно озвучить заранее (240 mp3 файлов) для нулевой latency. Стоимость через Azure ~\$40 total.

Для динамических ответов из БЗ TTS генерится на лету. Кэширование популярных ответов позволяет снизить расходы.

Особенности lip-sync на 6 языках:

- Русский/кыргызский — стандартные 15 виземы подходят
- Английский, немецкий, турецкий — тоже стандартные
- Китайский — тоны в аудио, но форма рта та же
- Одна библиотека виземы работает на все языки

Часть 21. Технологический стек проекта

Финальный технологический стек проекта:

Клиентская часть (Android):

Языки и фреймворки:

- Kotlin 2.0.21+
- Jetpack Compose (UI)
- Coroutines + Flow (асинхронность)
- Koin (DI)

Медиа:

- Media3 ExoPlayer 1.5.1 (воспроизведение видео)
- Media3 UI (PlayerView)
- Media3 Transformer (композиция видео на клиенте, опционально)

Аудио:

- AudioRecord с VOICE_COMMUNICATION source
- Silero VAD 2.0.5 (android-vad)
- WebRTC AEC (org.webrtc:google-webrtc)
- Google Cloud Speech-to-Text (или Yandex/Whisper)
- TTS: Azure или Google Cloud

Изображения:

- Coil для загрузки изображений в Compose

Network:

- Retrofit 2 + OkHttp
- kotlinx.serialization (JSON)
- WebSocket через OkHttp

Хранилище:

- DataStore (preferences)
- Room (история чата, кэш)
- WorkManager (фоновая синхронизация)

Опционально:

- SceneView 2.3.3 (если возвращаемся к 3D)
- Filament через SceneView
- ONNX Runtime + Mediapipe (для face detection, embeddings)

Kiosk mode:

- Device Policy Manager
- Lock Task Mode
- WindowInsetsControllerCompat для полноэкранного режима

Fuzzy matching:

- me.xdrop:fuzzywuzzy:1.4.0

Структура проекта (Clean Architecture):

```

nurai/
├── app/
│   ├── di/                                # Koin модули
│   │   ├── AudioModule.kt
│   │   ├── NetworkModule.kt
│   │   ├── StateMachineModule.kt
│   │   └── AnalyticsModule.kt

```

```
├── data/
│   ├── api/
│   │   ├── KnowledgeBaseApi.kt
│   │   ├── AnalyticsApi.kt
│   │   ├── ContentApi.kt
│   │   └── dto/
│   ├── repository/
│   │   ├── DialogRepository.kt
│   │   ├── ContentRepository.kt
│   │   └── AnalyticsRepository.kt
│   └── local/
│       ├── ContentCache.kt
│       └── SessionStorage.kt
├── domain/
│   ├── model/
│   │   ├── NuraiState.kt
│   │   ├── AnswerPayload.kt
│   │   ├── Language.kt
│   │   └── ErrorCode.kt
│   ├── usecase/
│   │   ├── ProcessUserQueryUseCase.kt
│   │   ├── HandleInterruptionUseCase.kt
│   │   ├── SessionLifecycleUseCase.kt
│   │   └── EmergencyModeUseCase.kt
│   └── filter/
│       ├── ProfanityFilter.kt
│       └── PiiDetector.kt
├── audio/
│   ├── pipeline/
│   │   ├── AudioInputPipeline.kt
│   │   ├── EchoCancellation.kt
│   │   └── NoiseGate.kt
│   ├── stt/
│   │   ├── SttEngine.kt
│   │   ├── GoogleSttEngine.kt
│   │   ├── VoskSttEngine.kt
│   │   └── LanguageDetector.kt
│   ├── tts/
│   │   └── TtsPlayer.kt
│   └── vad/
│       └── SileroVad.kt
├── presentation/
│   ├── MainActivity.kt
│   ├── NuraiViewModel.kt
│   ├── screens/
│   │   ├── IdleScreen.kt
│   │   ├── ListeningScreen.kt
│   │   ├── ProcessingScreen.kt
│   │   ├── SpeakingScreen.kt
│   │   ├── WebViewScreen.kt
│   │   ├── ErrorScreen.kt
│   │   └── EmergencyScreen.kt
│   └── components/
│       ├── AvatarVideoPlayer.kt
│       ├── AudioWaveform.kt
│       ├── LanguageGreeting.kt
│       ├── AttachmentChip.kt
│       └── LanguageSelector.kt
├── statemachine/
│   ├── StateMachine.kt
│   ├── transitions/
│   └── effects/
└── kiosk/
```

```
|  
└─ KioskLockManager.kt  
└─ DeviceAdminReceiver.kt
```

Веб vs нативное приложение:

Рассматривался вариант делать разработку на стороне веба (Three.js/Babylon.js в браузере на планшете). Отклонено из-за:

- Хуже производительность на слабом железе (WebGL медленнее чем нативный Filament)
- Chrome жрёт 400–700 МБ RAM
- Меньше контроля над устройством (kiosk mode)
- У нас одно фиксированное устройство, кроссплатформенность не нужна
- Уже есть готовый Android-проект с TTS/STT/репозиториями

Часть 22. Открытые вопросы для РМ

Открытые вопросы для РМ, требующие решения до начала активной разработки:

1. Аватар — подход к реализации

Варианты:

- A: Pre-recorded видео для FAQ (~30 записанных видео с актрисой) + D-ID для динамики из БЗ
- B: AI-генерация всей библиотеки через Kling 3.0 (~\$100 старт + \$30–100/мес)
- C: Live talking head сервис (D-ID \$30–300/мес) с одним reference фото

Рекомендация: гибрид B+C — AI-генерация статичной библиотеки + D-ID для динамики.

Требуемое решение: подтвердить подход и бюджет.

2. Микрофон — закупка железа

Варианты:

- A: ReSpeaker XVF3800 (\$70–100) — beamforming, AEC, шумоподавление
- B: Audfly AI Array (\$150–200) — специально для AI-киосков
- C: Замена устройства на 65TR3PN-B со встроенным микрофоном (+\$1500–2000)

Рекомендация: вариант A.

Требуемое решение: подтвердить закупку.

3. Дополнительная защита от посторонних голосов

Варианты:

- Только beamforming-микрофон (базово)
- + USB веб-камера для face detection (\$20–30)
- + Speaker identification (бесплатно, локально)

Рекомендация: полный пакет для шумного ТРЦ.

Требуемое решение: закупка USB-камеры.

4. Сервисы STT и TTS

Варианты:

- Google Cloud (STT + TTS) — универсально
- Azure TTS — есть виземы для lip-sync
- Yandex — топ для русского/кыргызского
- ElevenLabs — премиум голос для брендинга NURAI

Требуемое решение: выбор сервисов + бюджет.

5. API контракты с контакт-центром

Нужно от backend-команды КЦ:

- Спецификация /query endpoint
- Формат attachments (chips, links, cards, images)
- Endpoint для аналитики
- Endpoint синхронизации контента
- Whitelist доменов для WebView
- REST или WebSocket

Требуемое решение: созвон с командой КЦ.

6. Регламентные ответы на 6 языках

Варианты:

- Автоперевод через машину — быстро, дёшево, потеря стилистики
- Ручной перевод с адаптацией — правильное, дороже

Требуемое решение: кто отвечает за переводы, сроки.

7. Актриса или AI-генерация

Если идём по гибриднему варианту:

- Актриса выбрана?
- Когда планируется съёмка (если студийная)?
- Или сразу AI reference image?

Требуемое решение: подход к образу NURAI.

8. Аватар — 3D vs видео vs AI

Финальное решение по ТЗ:

- Видео — принято
- Изменится ли требование? История проекта показывает частые смены

Требуемое решение: зафиксировать финально.

9. Уточнение таймаутов

Моя трактовка:

- 5 сек — жёсткий timeout API КЦ
- 20 сек — общий SLA-KPI (от конца речи до начала озвучки)

- 5 сек тишины в Listening → возврат в Idle
- 60 сек после ответа → "У вас нет вопросов?"
- 120 сек тишины → прощаемся
- 30 сек в WebView → закрытие

Требуемое решение: подтверждение.

10. WebView whitelist доменов

Список разрешённых доменов:

- o.kg
- mobile.o.kg
- pay.o.kg
- ?

Требуемое решение: полный список от команды КЦ.

Часть 23. Ключевые ошибки и уроки проекта

Ключевые ошибки, обнаруженные и исправленные в процессе анализа проекта:

Ошибка 1: Не проверил T1 vs T2 в Avaturn

При первой попытке скачать модель был выбран T1 body type (для реалистичности), однако у T1 отсутствуют блендшейпы для анимации лица. Файл получился 4 МБ без морф-таргетов, что делало lip-sync невозможным.

Урок: всегда проверять экспорт через gltf-viewer.donmccurdy.com или modelviewer.dev перед интеграцией. Смотреть секцию Morph Targets в панели справа.

Ошибка 2: Ошибка в спецификациях LG-доски

Изначально я предполагал что у LG 65TR3DK-B: 4 GB RAM, 32 GB storage, Android 11, Mali-G31. По официальной странице LG на самом деле: 8 GB RAM, 64 GB storage, Android 13, Mali G52 MP2.

Урок: всегда сверяться с официальной документацией производителя, а не с моими предположениями. Особенно для не-мэйнстрим устройств типа digital signage.

Ошибка 3: Не знал про закрытие RPM

Ready Player Me был приобретён Netflix в декабре 2025 и закрывается 31 января 2026. Это ключевая информация которую я не имел. Пользователь сам поправил меня, что позволило переключиться на Avaturn.

Урок: технологические стеки быстро меняются, особенно AI/3D. Всегда проверять актуальность сервисов через веб-поиск. Не доверять только знаниям из тренинга.

Ошибка 4: Изменение требований — от Lottie к 3D к видео к AI

Проект прошёл 4 фазы смены концепции аватара:

1. Lottie 2D (первоначально)
2. 3D с SceneView (требование заказчика)
3. Видео (упрощение)
4. AI-генерация видео (оптимизация стоимости)

Каждая фаза добавляла работы. Урок: сразу закладывать в план 20-30% буфер на смену требований. Особенно при работе с корпоративными заказчиками где решения принимаются коллегиально.

Ошибка 5: Недооценка проблемы с микрофоном

Только на поздних этапах обнаружилось что у LG-доски нет встроенного микрофона. Это blocker для всего голосового флоу.

Урок: аппаратные требования проверять ПЕРВЫМИ, до архитектурных решений. Списком:

- Микрофон
- Камера (если нужна)
- Динамики
- Сетевое подключение
- Хранилище
- Порты для внешних устройств
- Возможность device owner mode

Ошибка 6: Не сверился с новыми AI-моделями

Изначально предлагал только D-ID/HeyGen для talking head. Не знал про Kling 3.0, Veo 3.1, Runway Gen-4.5 которые появились в 2026. После веб-поиска картина стала намного богаче.

Урок: AI-технологии обновляются ежемесячно. Для актуальных рекомендаций обязателен свежий веб-поиск.

Общие выводы по проекту:

1. Правильная последовательность разработки:
 - Утвердить hardware (устройство, микрофон, камера)
 - Утвердить архитектуру (state machine, потоки данных)
 - Утвердить визуальный дизайн (Figma, UX-флоу)
 - Утвердить контракты API (endpoints, форматы)
 - Утвердить контент (актриса, тексты, языки)
 - Начать разработку
2. Ключевые риски проекта:
 - Смена требований (высокий, история подтверждает)
 - Отсутствие микрофона (blocker)

- Кыргызский язык в STT (среднее качество)
- Стоимость AI-сервисов (D-ID, TTS) в долгосрочной перспективе
- Юридические вопросы с использованием AI-лица

3. Что помогло бы избежать проблем:

- Chek-list по hardware в начале
- Прототип аватара (реальный) до утверждения архитектуры
- Явное согласование стеков STT/TTS с бюджетом
- Согласование контрактов API до старта разработки
- Актриса выбрана до старта съёмок
- Резервный план на случай изменения требований

4. Что можно переиспользовать в следующих проектах O!:

- Клиентская архитектура (state machine, audio pipeline)
- Логика kiosk mode
- Регламентные ответы (Приложение 1)
- Многоязычность
- Beamforming микрофон + face detection комбо
- CDN контента + WorkManager sync

Проект NURAI — типичный пример современного conversational AI-киоска. Каждый компонент отдельно решаемый, но комбинация сложна и требует продуманной архитектуры.